

Shenzhen Dudumiao Technology Co., Ltd.

smokefire Fire, Smoke & Smoking Detection

Deployment Guide

Document version: V1.0

Applicable software version: 1.0.0

Updated: 2026-08-03

Contents

Document information	3
1. Delivery and requirements	3
1.1 Release assets	3
1.2 Host requirements	3
2. Download and verification	4
2.1 CPU bundle	4
2.2 GPU bundle	4
2.3 Verify files	4
3. CPU/GPU installation	5
3.1 Windows	5
3.2 Linux	5
3.3 Lifecycle and logs	5
4. Video integration modes	6
5. Existing go2rtc integration	6
5.1 How the upstream project integrates	6
5.2 Configure once	6
5.3 Validate automatic import	7
6. Bundled go2rtc and direct RTSP	7
6.1 Bundled go2rtc	7
6.2 Direct RTSP	8
7. First start and acceptance	8
8. Configuration and LAN access	8
9. Routine operation	9
10. Backup, restore, and upgrade	9

11. Troubleshooting	9
12. Handover checklist	10
Installation	10
Video and business acceptance	10

Important

smokefire is a video-AI assistance and alerting tool. It is not a certified fire alarm device and does not replace statutory fire protection, inspections, emergency procedures, or human verification. False positives and missed detections remain possible.

Document information

Item	Value
Audience	Deployment engineers, administrators, and site operators
Delivery	Source-free offline Docker bundles for CPU and NVIDIA GPU
Hosts	Windows 10/11, Windows Server, mainstream x86_64 Linux
Video input	Direct RTSP, bundled go2rtc, or bulk sync from an existing go2rtc
Default URL	http://127.0.0.1:8600
Project	https://github.com/newtv-ai/smokefire

1. Delivery and requirements

1.1 Release assets

GitHub Release [v1.0.0](#) provides two source-free offline deliveries:

- [smokefire-deploy-1.0.0-cpu.zip](#): complete CPU bundle;
- [smokefire-deploy-1.0.0-gpu.zip](#): GPU configuration, models, scripts, and manuals;
- [smokefire-images.tar.gz.partNN](#): all GPU image parts, required with the GPU bundle;
- [SHA256SUMS-v1.0.0.txt](#): checksums for Release assets.

Each delivery includes prebuilt smokefire and go2rtc Docker images, [fire_smoke_v5.pt](#), [smoking_v4.pt](#), Compose files, Windows/Linux scripts, and all six Chinese/English PDFs. The target host does not need Python, Git, compilers, or GitHub access after installation.

1.2 Host requirements

Item	CPU bundle	GPU bundle
CPU	x86_64, 4+ cores recommended	x86_64, 4+ cores recommended
RAM	8 GB minimum, 16 GB recommended	16 GB minimum
GPU	Not required	NVIDIA GPU, 8 GB+ VRAM recommended
Driver	-	Host driver must support the bundled CUDA 12.8 runtime
Docker	Engine/Desktop 24+ and Compose v2	Same, plus NVIDIA Container Toolkit
Free disk	20 GB plus event data	50 GB plus event data

Item	CPU bundle	GPU bundle
Cameras	RTSP; H.264 recommended	Same

The CPU build suits pilots and smaller sites. The GPU build suits higher concurrency, but actual capacity still depends on codec, resolution, sampling interval, recording, network conditions, and the target hardware. Validate the planned stream count with real feeds.

2. Download and verification

The repository is currently Private. Authorized delivery users must sign in to GitHub and open Release v1.0.0 at:

```
https://github.com/newtv-ai/smokefire
```

2.1 CPU bundle

Download and extract `smokefire-deploy-1.0.0-cpu.zip`.

2.2 GPU bundle

Download `smokefire-deploy-1.0.0-gpu.zip` and every GPU image part. Extract the bundle and place the parts under `images/`:

```
smokefire/  
images/  
  smokefire-images.tar.gz.part01  
  smokefire-images.tar.gz.part02  
  ...
```

The start script assembles the parts in filename order, loads the Docker images, and removes only its temporary assembled file. Original parts remain intact.

2.3 Verify files

Windows:

```
powershell -ExecutionPolicy Bypass -File .\verify.ps1
```

Linux:

```
chmod +x verify.sh start.sh stop.sh configure-go2rtc.sh  
./verify.sh
```

Fixed model checksums:

File	Size	SHA-256
models/fire_smoke_v5.pt	44,014,233 bytes	b5248d55c5d90e341bc4e537887d4bce4b9af1bd0330ddd87825294750a1e216
models/smoking_v4.pt	44,025,689 bytes	0ea7b66c2105f1898f56f828b18ffb01934fb1f47a795192e8867ae8fcb128dd

Do not continue after any checksum failure. Download the affected file again.

3. CPU/GPU installation

3.1 Windows

Install and start Docker Desktop using Linux containers. The GPU build also requires a working NVIDIA driver and Docker GPU support.

```
docker version
docker compose version
```

After the offline image has been imported by the start script, validate the GPU build with:

```
nvidia-smi
docker run --rm --gpus all smokefire:1.0.0-gpu python -c "import torch;
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

Start the service:

```
powershell -ExecutionPolicy Bypass -File .\start.ps1
```

3.2 Linux

Install Docker Engine and Compose v2. For the GPU build, also install the NVIDIA driver and NVIDIA Container Toolkit.

```
docker version
docker compose version
nvidia-smi          # GPU bundle only
chmod +x verify.sh start.sh stop.sh configure-go2rtc.sh
./start.sh
```

The script verifies the delivery, selects the CPU or GPU image from `.env`, loads smokefire and go2rtc offline, starts the containers, and waits for `/api/health/ready`.

3.3 Lifecycle and logs

```
docker compose ps
docker compose logs -f smokefire
```

Use `stop.ps1` on Windows or `./stop.sh` on Linux. Stopping preserves business data. Never run `docker compose down -v`, which removes named volumes.

4. Video integration modes

Choose one mode for the site:

Mode	Best fit	Configuration effort	Data path
A. Direct RTSP	A few cameras and no go2rtc	One URL per camera	Camera/NVR -> smokefire
B. Bundled go2rtc	Cameras are managed in smokefire; detection and recording should share one producer	One URL per camera in smokefire	Camera -> bundled go2rtc -> detection/recording
C. Existing go2rtc	The customer already manages feeds in go2rtc	One API base plus one RTSP base	Existing go2rtc -> automatic import -> smokefire

For an existing go2rtc site, use mode C. Do not copy every camera RTSP URL into smokefire.

The provided configuration script changes only `.env` in the deployment directory. If the service is already running, it recreates the smokefire container while preserving business data.

5. Existing go2rtc integration

5.1 How the upstream project integrates

smokefire retains the upstream system's bulk synchronization approach:

```

Customer go2rtc GET /api/streams
      |
      v
smokefire synchronizes main-stream names and states
      |
      v
rtsp://customer-go2rtc:8554/<URL-encoded-stream-name>
      |
      v
AI detection, recording, preview, and events

```

The synchronizer only reads `GET /api/streams`; it never calls create, update, or delete operations on the customer's go2rtc. The first sync registers all main streams automatically. When a matching main stream exists, names ending in `_sub`, `-sub`, or `_sd` are treated as substreams and skipped to prevent duplicates. At the default 300-second interval, new streams are added, missing streams are disabled, and returning streams are enabled again.

5.2 Configure once

If go2rtc runs on the same Docker host, do not use `127.0.0.1` from inside the smokefire container. Use `host.docker.internal`:

```
.\configure-go2rtc.ps1 -Mode upstream `
  -Api http://host.docker.internal:1984 `
  -Rtsp rtsp://host.docker.internal:8554
```

Linux:

```
./configure-go2rtc.sh upstream \
  http://host.docker.internal:1984 \
  rtsp://host.docker.internal:8554
```

For go2rtc on another LAN host, use its LAN address, for example:

```
API http://192.168.10.20:1984
RTSP rtsp://192.168.10.20:8554
```

Equivalent `.env` entries:

```
SMOKEFIRE_GO2RTC_ENABLED=false
SMOKEFIRE_UPSTREAM_GO2RTC_ENABLED=true
SMOKEFIRE_UPSTREAM_GO2RTC_API=http://192.168.10.20:1984
SMOKEFIRE_UPSTREAM_GO2RTC_RTSP=rtsp://192.168.10.20:8554
SMOKEFIRE_UPSTREAM_GO2RTC_SYNC_INTERVAL_SEC=300
```

5.3 Validate automatic import

- 1 Open <http://<go2rtc-host>:1984/api/streams> and confirm it returns the expected feeds.
- 2 Run the configuration script and start smokefire.
- 3 Open Camera Management and confirm main streams appear automatically with the Upstream badge.
- 4 Confirm matching substreams were not duplicated.
- 5 Validate at least one preview, AI inference result, and event recording.
- 6 Add a test stream and confirm the next sync imports it; remove and restore it to validate automatic disable/re-enable.

Keep existing go2rtc stream names unique and stable. The synchronizer owns imported source URLs; do not create manual cameras with the same names.

6. Bundled go2rtc and direct RTSP

6.1 Bundled go2rtc

Use this when the site has no go2rtc but wants detection and recording to share one upstream camera producer:

```
.\configure-go2rtc.ps1 -Mode builtin
```



```
./configure-go2rtc.sh builtin
```

Continue adding camera RTSP URLs in Camera Management. smokefire creates internal `sf-camN` aliases automatically, and both AI and recording consume the bundled go2rtc output.

6.2 Direct RTSP

```
.\configure-go2rtc.ps1 -Mode direct
```

```
./configure-go2rtc.sh direct
```

Add camera or NVR RTSP URLs individually in Camera Management. This is the simplest mode, but AI and recording may create separate upstream sessions. Prefer go2rtc as the stream count grows.

7. First start and acceptance

Open <http://127.0.0.1:8600> on the service host. Initial model loading may take several minutes.

Endpoint	Purpose	Expected
/api/health/live	Process liveness	HTTP 200
/api/health/ready	Models, scheduler, recording, disk, and upstream sync	HTTP 200
/api/status	Runtime and notification metrics	JSON response

Minimum acceptance:

- 1 Live Wall, Event Center, and Camera Management open successfully.
- 2 Video is connected through the selected mode; an existing-go2rtc deployment proves bulk automatic import.
- 3 Cameras become Online with correct names, images, and timestamps.
- 4 Authorized test material exercises both the fire/smoke and smoking models.
- 5 Event details contain snapshot, class, confidence, time, and recording.
- 6 Confirm True, Mark False Positive, reconnect behavior, restart, and data persistence work.
- 7 Run the planned stream count and record throughput, latency, GPU/CPU, disk, and network behavior.

Installation success is not production acceptance. See the Model Capability Test Guide for model boundaries and site test design.

8. Configuration and LAN access

The Web port binds to `127.0.0.1:8600` by default. For LAN access, an engineer must set the bind address and allowed hostnames in `.env`, then restart.

Setting	Default	Purpose
SMOKEFIRE_IMAGE	Set by CPU/GPU bundle	Application image
SMOKEFIRE_ALLOWED_HOSTS	localhost,127.0.0.1	Accepted Web Host values
SMOKEFIRE_GO2RTC_ENABLED	false	Enables bundled go2rtc fan-out
SMOKEFIRE_UPSTREAM_GO2RTC_ENABLED	false	Enables existing-go2rtc synchronization
SMOKEFIRE_UPSTREAM_GO2RTC_API	Empty	Existing go2rtc API base
SMOKEFIRE_UPSTREAM_GO2RTC_RTSP	Empty	Existing go2rtc RTSP base
SMOKEFIRE_INFER_WORKERS	4	Inference workers
SMOKEFIRE_INFER_BATCH_MAX	8	Maximum inference batch
SMOKEFIRE_STOP_GRACE_PERIOD	180s	Graceful shutdown period

RTSP URLs may contain credentials. Restrict access to the deployment directory, backups, and terminal history, and redact full credentials from screenshots and support tickets.

9. Routine operation

```
docker compose ps
docker compose logs --tail 200 smokefire
curl http://127.0.0.1:8600/api/health/ready
```

Daily checks should cover camera status, latest successful upstream sync, preview freshness, event playback, notification queues, and disk high-water state. In existing-go2rtc mode, compare the number of go2rtc main streams with Upstream cameras in smokefire.

10. Backup, restore, and upgrade

Business data is stored in the `smokefire-data` named volume. Stop the service, record the version, and copy the entire `/app/data` tree. A complete backup includes camera records, the SQLite database, event videos, snapshots, false-positive feedback, and notification outbox. The customer's existing go2rtc configuration remains the customer's backup responsibility and is not stored in the smokefire volume.

Restore data to the matching software version, then validate health, cameras, historical events, recordings, feedback, and upstream sync.

Before upgrading, verify that the backup is readable. Download and verify the new bundle, stop the old version, start from a new directory, and run minimum acceptance. Roll back to the prior image and backup if validation fails.

11. Troubleshooting

Symptom	First check	Action
Docker command unavailable	Docker Desktop/Engine	Start the engine and retry docker version
GPU container unavailable	Driver, Container Toolkit, Docker GPU support	Pass nvidia-smi and container CUDA checks first
GPU image part missing	images/ , filenames, SHA-256	Download all parts without renaming them
Readiness remains false	Logs, models, disk, upstream sync	Run docker compose logs --tail 300 smokefire
Existing go2rtc imports nothing	API URL, container reachability to 1984, /api/streams	Use host.docker.internal on the same host or the LAN IP remotely
Fewer imports than go2rtc names	Main/substream naming	_sub , -sub , and _sd matches are deliberately deduplicated
Upstream camera becomes disabled	Feed disappeared or sync failed	Restore the feed; the next successful sync re-enables it
Camera keeps reconnecting	RTSP 8554, encoded stream name, source state	Test the same go2rtc RTSP URL from the deployment host
Too many or no alerts	Threshold, target pixels, scene interference	Replay representative positive and negative samples; do not lower thresholds blindly
Event has no video	Disk, warm-up, source stability	Inspect disk status and FFmpeg logs

12. Handover checklist

Installation

- ☐ The complete CPU or GPU offline delivery was selected and verified.
- ☐ smokefire, go2rtc, and both model checks passed.
- ☐ Start, stop, restart, and host reboot recovery were tested.
- ☐ [/api/health/live](#) and [/api/health/ready](#) return normally.
- ☐ Install directory, data volume, backup location, and version 1.0.0 are recorded.
- ☐ The repository remains Private and delivery users have authorized access.

Video and business acceptance

- ☐ Direct RTSP, bundled go2rtc, or existing go2rtc mode is explicitly selected.
- ☐ Existing-go2rtc sites validated one-time setup, bulk main-stream import, addition, removal, and recovery.
- ☐ Planned cameras have correct names, time, image, inference, and recording.
- ☐ Both models were tested with representative positive and negative samples.
- ☐ Snapshots, recordings, Confirm True, false-positive archive, and feedback export work.
- ☐ Planned capacity, reconnect behavior, and continuous operation were tested.

- ☐ Users received the User Operation Manual and Model Capability Test Guide and completed handover training.

Prepared by: Shenzhen Dudumiao Technology Co., Ltd. Project:
<https://github.com/newtv-ai/smokefire>